PARUL UNIVERSITY

Faculty of Computer Applications

LAB MANUAL

05010107DS06

Open Source Software and Tools (Practical)

Program	BCA (Hons)
Semester	7th Semester
Credits	1
Lab Hours	2 Hours/Week
Internal Marks	20
External Marks	30
Total Marks	50

Academic Year 2025-26

Course Overview

Prerequisites

- Basic knowledge of Software Development Life Cycle (SDLC)
- Programming fundamentals in Java or Python
- Fundamentals of web technologies: HTML and CSS

Course Outcomes

After completing this lab course, students will be able to:

- Understand fundamentals of software testing and testing life cycle
- Understand different testing levels and techniques
- Know the importance of automation in testing with testing tools
- Understand defect life cycle and management
- Understand testing optimization tools and software quality metrics

Tools Required

- PHP 8.x (or 7.4+)
- Composer (PHP dependency manager)
- Laravel Framework (v10 or v11)
- XAMPP or WAMP (for local web server and MySQL)
- Visual Studio Code or any preferred IDE
- Web browser (Chrome / Firefox)
- WordPress CMS

SECTION A: Laravel Framework

Practical 1: Install Laravel using Composer

Aim	Install Composer and Laravel on a local machine.
Duration	1 Lab Session (2 Hours)
Tools	PHP 8+, Composer, Terminal / Command Prompt

Theory

Laravel is a free, open-source PHP web framework designed for building modern web applications following the MVC (Model-View-Controller) architectural pattern. Composer is the dependency manager for PHP that Laravel uses to manage packages and libraries.

Step-by-Step Instructions

Step 1: Install PHP

1. Download XAMPP from https://www.apachefriends.org and install it.
2. After installation, open a terminal and verify PHP is installed:

```
php -v
```

3. You should see output like: PHP 8.x.x (cli)

Step 2: Install Composer

4. Visit https://getcomposer.org/download/ and download the installer for your OS.
5. For Windows: Run the Composer-Setup.exe installer. Make sure PHP is added to PATH.
6. For Linux/Mac: Run the following commands in your terminal:

```
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
```

7. Verify Composer is installed:

```
composer --version
```

Step 3: Install Laravel Installer (Global)

8. Run the following Composer command to install Laravel globally:

```
composer global require laravel/installer
```

9. Add the Composer global bin directory to your system PATH.
10. Verify Laravel CLI is available:

```
laravel --version
```

Step 4: Create a New Laravel Project

11. Navigate to your desired project directory (e.g., htdocs in XAMPP):

```
cd C:\xampp\htdocs
```

12. Create a new Laravel project named myapp:

```
laravel new myapp
# OR using Composer directly:
composer create-project laravel/laravel myapp
```

13. Navigate into the project folder:

```
cd myapp
```

Step 5: Start the Development Server

14. Run the built-in Laravel development server:

```
php artisan serve
```

15. Open your browser and visit: http://127.0.0.1:8000
16. You should see the Laravel welcome page.

Expected Output: Laravel welcome page displayed at http://127.0.0.1:8000

Common Errors and Solutions

Error: 'composer' is not recognized as an internal command => Ensure Composer is added to system PATH and restart terminal.

Error: PHP version conflict => Ensure PHP 8.0 or higher is installed and selected in your PATH.

Viva Questions

17. What is Composer? How is it different from npm?
18. What is the role of the artisan command in Laravel?
19. What PHP version does Laravel 10 require?
20. What does the vendor folder contain in a Laravel project?

Practical 2: Create a New Laravel Project: Basic Laravel Authenticator

Aim	Set up a Laravel project with built-in authentication (login, register, logout).
Duration	1 Lab Session (2 Hours)
Tools	Laravel, Composer, Breeze/UI Package, Browser

Theory

Laravel provides multiple starter kits for authentication. Laravel Breeze is the simplest option that includes login, registration, email verification, and password reset. It uses Blade templates with Tailwind CSS.

Step-by-Step Instructions

Step 1: Create a New Project

```
composer create-project laravel/laravel auth-demo
cd auth-demo
```

Step 2: Configure the Database

21. Open the .env file in your project root.
22. Update the database settings:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=auth_demo
DB_USERNAME=root
DB_PASSWORD=
```

23. Create the database auth_demo in phpMyAdmin (via XAMPP).

Step 3: Install Laravel Breeze

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
```

Step 4: Run Migrations

```
php artisan migrate
```

This creates the users table and other default tables.

Step 5: Serve and Test

```
php artisan serve
```

24. Visit http://127.0.0.1:8000/register to create an account.
25. After registering, you will be redirected to the dashboard.
26. Test the login at http://127.0.0.1:8000/login
27. Test the logout functionality from the navigation bar.

Expected Output: Working register, login, logout, and dashboard pages with Tailwind CSS styling.

Viva Questions

28. What is Laravel Breeze? How is it different from Laravel Jetstream?
29. What does php artisan migrate do?
30. Where are the authentication routes defined in a Breeze project?
31. What is the purpose of the .env file?

Practical 3: Create Routes for Different Pages (Home, About, Contact)

Aim	Define and configure routes for multiple pages in Laravel.
Duration	1 Lab Session
Tools	Laravel, routes/web.php, Artisan

Theory

In Laravel, routes are defined in the routes/web.php file. Routes map a URL to a controller method or a closure. Laravel supports GET, POST, PUT, DELETE and other HTTP methods.

Step-by-Step Instructions

Step 1: Open routes/web.php

Navigate to your project and open routes/web.php

Step 2: Define Routes

```
<?php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('home');
})->name('home');

Route::get('/about', function () {
    return view('about');
})->name('about');

Route::get('/contact', function () {
    return view('contact');
})->name('contact');
```

Step 3: Create Blade View Files

Create the following files in the resources/views/ folder:

- home.blade.php
- about.blade.php
- contact.blade.php

Step 4: Add Content to Each View

Example: resources/views/home.blade.php

```
<!DOCTYPE html>
<html>
<head><title>Home</title></head>
<body>
    <h1>Welcome to the Home Page</h1>
    <nav>
        <a href="{{ route('home') }}">Home</a> |
        <a href="{{ route('about') }}">About</a> |
        <a href="{{ route('contact') }}">Contact</a>
    </nav>
</body>
</html>
```

Repeat similarly for about.blade.php and contact.blade.php with appropriate content.

Step 5: Test the Routes

32. Start the server: php artisan serve
33. Visit http://127.0.0.1:8000/ for Home
34. Visit http://127.0.0.1:8000/about for About
35. Visit http://127.0.0.1:8000/contact for Contact

Step 6: List All Routes

```
php artisan route:list
```

Expected Output: Three separate pages accessible via URL, each showing their respective content.

Viva Questions

36. What is the difference between a named route and an anonymous route?
37. What HTTP methods does Laravel support in routes?
38. What is the purpose of Route::group()?

Practical 4: Create Corresponding View Files for Each Route

Aim	Create and link Blade view templates for each defined route using layouts.
Duration	1 Lab Session
Tools	Laravel Blade Engine, resources/views/

Theory

Laravel uses the Blade templating engine. Blade allows template inheritance using @extends and @section directives, making it easy to create consistent layouts across pages.

Step-by-Step Instructions

Step 1: Create a Master Layout

Create resources/views/layouts/app.blade.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>@yield('title', 'My App')</title>
    <style>
        body { font-family: Arial; margin: 40px; background: #f5f5f5; }
        nav a { margin-right: 15px; text-decoration: none; color: #1F4E79; }
    </style>
</head>
<body>
    <nav>
        <a href="{{ route('home') }}">Home</a>
        <a href="{{ route('about') }}">About</a>
        <a href="{{ route('contact') }}">Contact</a>
    </nav>
    <hr>
    <div class="content">
        @yield('content')
    </div>
</body>
</html>
```

Step 2: Update Each View to Extend Layout

resources/views/home.blade.php:

```
@extends('layouts.app')
@section('title', 'Home')
```

```
@section('content')
    <h1>Welcome to Home Page</h1>
    <p>This is the home page of our Laravel application.</p>
@endsection
```

resources/views/about.blade.php:

```
@extends('layouts.app')
@section('title', 'About Us')
@section('content')
    <h1>About Us</h1>
    <p>We are a team of passionate web developers.</p>
@endsection
```

resources/views/contact.blade.php:

```
@extends('layouts.app')
@section('title', 'Contact')
@section('content')
    <h1>Contact Us</h1>
    <p>Email us at: contact@example.com</p>
@endsection
```

Step 3: Test All Pages

39. Run php artisan serve
40. Visit each page URL and confirm the consistent navigation layout appears on all pages.

Expected Output: All pages share the same navigation bar and layout via the master template.

Viva Questions

41. What is @yield and @section in Blade?
42. How does template inheritance work in Laravel Blade?
43. What is the difference between @include and @extends?

Practical 5: Pass Data from Routes to Views

Aim	Pass dynamic data from routes to Blade view files and display it.
Duration	1 Lab Session
Tools	Laravel routes/web.php, Blade Views

Theory

Data can be passed from routes or controllers to views using the compact() function, with() method, or by directly passing an associative array as the second argument of the view() function.

Step-by-Step Instructions

Method 1: Pass Data Using Array

```
// In routes/web.php
Route::get('/profile', function () {
    $name = "John Doe";
    $age  = 22;
    $skills = ['PHP', 'Laravel', 'MySQL'];
    return view('profile', ['name' => $name, 'age' => $age, 'skills' =>
$skills]);
});
```

Method 2: Pass Data Using compact()

```
Route::get('/profile', function () {
    $name = "John Doe";
    $age  = 22;
    $skills = ['PHP', 'Laravel', 'MySQL'];
    return view('profile', compact('name', 'age', 'skills'));
});
```

Method 3: Pass Data Using with()

```
Route::get('/profile', function () {
    return view('profile')
        ->with('name', 'John Doe')
        ->with('age', 22);
});
```

Step 2: Create the Profile View

Create resources/views/profile.blade.php:

```
@extends('layouts.app')
@section('title', 'Profile')
@section('content')
    <h2>Student Profile</h2>
    <p><strong>Name:</strong> {{ $name }}</p>
    <p><strong>Age:</strong> {{ $age }}</p>
    <h3>Skills:</h3>
    <ul>
        @foreach($skills as $skill)
            <li>{{ $skill }}</li>
        @endforeach
    </ul>
@endsection
```

Step 3: Test

44. Run php artisan serve
45. Visit http://127.0.0.1:8000/profile
46. Verify name, age, and skills are displayed correctly.

Expected Output: Profile page displaying dynamic data: name, age, and a list of skills.

Viva Questions

47. What are the different ways to pass data from a route to a view?
48. What does the {{ }} syntax do in Blade? Is it safe from XSS?
49. What is the difference between {{ }} and {!! !!} in Blade?

Practical 6: Create Migrations for Database Tables

Aim	Create and run database migrations for users and posts tables in Laravel.
Duration	1 Lab Session
Tools	Laravel Artisan, MySQL (XAMPP), .env Configuration

Theory

Migrations in Laravel are like version control for your database. They allow you to define your database structure in PHP and run changes using artisan commands. Each migration file contains an up() method (to apply changes) and a down() method (to reverse them).

Step-by-Step Instructions

Step 1: Configure Database Connection

Update .env file:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_db
DB_USERNAME=root
DB_PASSWORD=
```

50. Start XAMPP and create the database laravel_db in phpMyAdmin.

Step 2: Create Posts Migration

```
php artisan make:migration create_posts_table
```

Step 3: Define the Posts Table Schema

Open the newly created migration in database/migrations/ and update the up() method:

```
public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->string('title');
        $table->text('content');
        $table->string('status')->default('draft');
        $table->timestamps();
    });
}
```

```
public function down(): void
{
    Schema::dropIfExists('posts');
}
```

Step 4: Run Migrations

```
php artisan migrate
```

This creates all tables (including the default users table) in your database.

Step 5: Verify in phpMyAdmin

51. Open XAMPP and start Apache and MySQL.
52. Visit http://localhost/phpmyadmin
53. Open laravel_db and confirm the tables are created: users, posts, migrations, etc.

Useful Artisan Commands

```
php artisan migrate:rollback   # Undo last batch of migrations
php artisan migrate:refresh    # Rollback all and re-migrate
php artisan migrate:status     # Show migration status
```

Expected Output: Tables users, posts, and others created in the MySQL database.

Viva Questions

54. What is the purpose of a migration in Laravel?
55. What does php artisan migrate:rollback do?
56. What is the difference between timestamps() and softDeletes()?
57. How do you create a foreign key relationship in a migration?

Practical 7: Perform CRUD Operations on Database Records

Aim	Implement Create, Read, Update, Delete (CRUD) operations for a Posts module.
Duration	2 Lab Sessions
Tools	Laravel MVC, Eloquent ORM, Blade, MySQL

Theory

CRUD operations are the four basic functions of persistent storage. Laravel uses Eloquent ORM to interact with the database using PHP models. A resourceful controller provides all CRUD methods automatically.

Step-by-Step Instructions

Step 1: Create Post Model and Controller

```
php artisan make:model Post -mcr
```

This creates: Model (Post.php), Migration, and a Resource Controller.

Step 2: Define Fillable Fields in Model

Open app/Models/Post.php:

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    protected $fillable = ['title', 'content', 'status'];
}
```

Step 3: Define Resource Route

In routes/web.php:

```
use App\Http\Controllers\PostController;
Route::resource('posts', PostController::class);
```

Step 4: Implement Controller Methods

Open app/Http/Controllers/PostController.php:

```
// INDEX - List all posts
public function index() {
```

```
    $posts = Post::latest()->get();
    return view('posts.index', compact('posts'));
}

// CREATE - Show form
public function create() {
    return view('posts.create');
}

// STORE - Save to DB
public function store(Request $request) {
    Post::create($request->validate([
        'title'   => 'required|max:255',
        'content' => 'required',
    ]));
    return redirect()->route('posts.index')->with('success', 'Post created!');
}

// EDIT - Show edit form
public function edit(Post $post) {
    return view('posts.edit', compact('post'));
}

// UPDATE - Save changes
public function update(Request $request, Post $post) {
    $post->update($request->validate([
        'title'   => 'required|max:255',
        'content' => 'required',
    ]));
    return redirect()->route('posts.index')->with('success', 'Post updated!');
}

// DESTROY - Delete
public function destroy(Post $post) {
    $post->delete();
    return redirect()->route('posts.index')->with('success', 'Post deleted!');
}
```

Step 5: Create Blade Views

Create the folder resources/views/posts/ and the following files:

- index.blade.php — List all posts with Edit and Delete buttons
- create.blade.php — Form to create a new post
- edit.blade.php — Pre-filled form to edit a post

Sample create.blade.php:

```
@extends('layouts.app')
@section('content')
<h2>Create Post</h2>
```

```
<form method="POST" action="{{ route('posts.store') }}">
    @csrf
    <input type="text" name="title" placeholder="Title" required><br>
    <textarea name="content" placeholder="Content" required></textarea><br>
    <button type="submit">Save Post</button>
</form>
@endsection
```

Step 6: Test All CRUD Operations

58. Visit http://127.0.0.1:8000/posts to list posts.
59. Click Create New Post to add a record.
60. Click Edit to modify a record.
61. Click Delete to remove a record.

Expected Output: Fully working CRUD interface: create, view, edit, and delete posts from the database.

Viva Questions

62. What is Eloquent ORM? How is it different from raw SQL?
63. What does Route::resource() generate?
64. What is @csrf and why is it required in forms?
65. What is the difference between save() and create() in Eloquent?

Practical 8: Create and Deploy Demo Web Application using Laravel

Aim	Build and deploy a complete demo web application integrating PHP with Laravel.
Duration	2 Lab Sessions
Tools	Laravel, MySQL, XAMPP, PHP, Blade, Bootstrap (optional)

Theory

This practical consolidates all previous Laravel practicals into a complete mini-project. Students will integrate routing, views, database migrations, authentication, and CRUD into a deployable application.

Project: Student Task Manager

Step 1: Create New Project with Auth

```
composer create-project laravel/laravel task-manager
cd task-manager
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
```

Step 2: Create Tasks Migration and Model

```
php artisan make:model Task -mcr
```

Migration schema in database/migrations/..._create_tasks_table.php:

```
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->string('title');
    $table->text('description')->nullable();
    $table->enum('priority', ['low', 'medium', 'high'])->default('medium');
    $table->boolean('completed')->default(false);
    $table->timestamps();
});
```

Step 3: Set Fillable and Relationships

```
// Task.php
protected $fillable = ['title', 'description', 'priority', 'completed'];

// User.php - add relationship
```

```
public function tasks() {
    return $this->hasMany(Task::class);
}
```

Step 4: Add Auth-Protected Task Routes

```
Route::middleware('auth')->group(function () {
    Route::resource('tasks', TaskController::class);
});
```

Step 5: Implement Task Controller

Ensure all controller methods use auth()->user()->tasks() to scope operations to the logged-in user.

Step 6: Run and Test

```
php artisan migrate
php artisan serve
```

66. Register an account, login, and test task creation, editing, completion toggle, and deletion.

Expected Output: A fully functional authenticated task management web application.

Deployment on XAMPP

67. Copy the project folder to xampp/htdocs/
68. Update .env for production settings and run php artisan config:cache
69. Access via http://localhost/task-manager/public

Viva Questions

70. How does middleware protect routes in Laravel?
71. What is MVC and how does Laravel implement it?
72. What steps are needed to deploy a Laravel app on a shared hosting?

SECTION B: WordPress CMS

Practical 9: Install WordPress on a Local Server Using XAMPP/WAMP

Aim	Download, install, and configure WordPress on a local server environment.
Duration	1 Lab Session
Tools	XAMPP or WAMP, WordPress, phpMyAdmin, Browser

Theory

WordPress is the world's most popular open-source Content Management System (CMS). It is built with PHP and MySQL. XAMPP (for cross-platform) or WAMP (Windows only) provides a local Apache, PHP, and MySQL environment for running WordPress without an internet hosting plan.

Step-by-Step Instructions

Step 1: Start XAMPP Services

73. Open XAMPP Control Panel.
74. Start Apache and MySQL modules.
75. Verify both show 'Running' status with green indicators.

Step 2: Create a Database

76. Open browser and visit: http://localhost/phpmyadmin
77. Click Databases tab.
78. Enter database name: wordpress_db
79. Select Collation: utf8_general_ci
80. Click Create.

Step 3: Download WordPress

81. Visit https://wordpress.org/download/
82. Download the latest WordPress .zip file.
83. Extract the zip file.
84. Copy the extracted 'wordpress' folder to: C:\xampp\htdocs\
85. Rename the folder to mysite (optional).

Step 4: Run WordPress Installer

86. Open browser and visit: http://localhost/mysite

87. Select your language and click Continue.
88. On the next screen, click Let's go!
89. Enter database details:

```
Database Name: wordpress_db
Username:      root
Password:      (leave blank for XAMPP default)
Database Host: localhost
Table Prefix:  wp_
```

90. Click Submit, then Run the Installation.

Step 5: Complete Installation

91. Fill in the Site Title, Admin Username, Password, and Email.
92. Click Install WordPress.
93. After success, click Log In.
94. Login with your admin credentials to access the WordPress Dashboard.

Expected Output: WordPress dashboard accessible at http://localhost/mysite/wp-admin

Viva Questions

95. What is a CMS? Give examples other than WordPress.
96. What database does WordPress use by default?
97. What is the significance of the wp-config.php file?
98. What is the difference between XAMPP and WAMP?

Practical 10: Customize the Site Title, Tagline, and Site Icon

Aim	Customize WordPress site identity settings including title, tagline, and favicon.
Duration	30 Minutes
Tools	WordPress Admin Dashboard, Customizer

Step-by-Step Instructions

Step 1: Access WordPress Customizer

99. Login to WordPress Dashboard at http://localhost/mysite/wp-admin
100. Go to Appearance > Customize from the left menu.
101. The live WordPress Customizer will open.

Step 2: Update Site Identity

102. Click on Site Identity in the Customizer panel.
103. Update the Site Title (e.g., 'My Lab Website').
104. Update the Tagline (e.g., 'Learning WordPress Practically').
105. Toggle Display Site Title and Tagline ON.
106. Click Publish/Save to apply changes.

Step 3: Upload a Site Icon (Favicon)

107. In the Site Identity section, scroll down to find Site Icon.
108. Click Select site icon.
109. Upload a square image (minimum 512x512 pixels, PNG or JPG).
110. Crop if prompted, then click Select.
111. Click Publish to save.

Step 4: Alternatively via Settings

112. Go to Settings > General in the Dashboard.
113. Update Site Title and Tagline fields.
114. Scroll down and click Save Changes.

Expected Output: Browser tab now shows the custom site title and favicon. The tagline is visible on the homepage.

Viva Questions

115. Where is the site identity stored in WordPress?
116. What is a favicon and what size should it be?
117. What is the WordPress Customizer used for?

Practical 11: Create and Publish a Blog Post with Categories and Tags

Aim	Create, categorize, tag, and publish a blog post in WordPress.
Duration	1 Lab Session
Tools	WordPress Admin Dashboard, Block Editor (Gutenberg)

Theory

WordPress uses a block-based editor called Gutenberg. Posts are for time-based content (blogs), while Pages are for static content. Categories are hierarchical and group posts broadly; Tags are flat keywords for specific topics.

Step-by-Step Instructions

Step 1: Create Categories

118. Go to Posts > Categories in the Dashboard.
119. Add new category: Name = Technology, Slug = technology
120. Click Add New Category.
121. Add another: Name = Tutorial

Step 2: Create a New Post

122. Go to Posts > Add New.
123. Enter the post title: 'Introduction to Open Source Software'
124. Click in the body area and start typing the post content.
125. Use the + icon to add blocks: Paragraph, Heading, List, Quote, etc.

Step 3: Assign Category

126. In the right panel, find the Categories section.
127. Check the Technology category.

Step 4: Add Tags

128. In the right panel, find the Tags section.
129. Type a tag (e.g., 'opensource') and press Enter.
130. Add more tags: 'linux', 'php', 'web'

Step 5: Set Featured Image (Optional)

131. Scroll down in the right panel and click Set Featured Image.
132. Upload or select an image from the media library.

Step 6: Publish the Post

133. Click Publish at the top right.
134. Confirm the publication in the popup.
135. Click View Post to see the live post.

Expected Output: Published blog post visible on the site frontend, with category and tags displayed.

Viva Questions

136. What is the difference between a Post and a Page in WordPress?
137. What is the difference between Categories and Tags?
138. What is Gutenberg in WordPress?
139. What does 'Draft', 'Pending Review', and 'Published' status mean?

Practical 12: Upload and Insert Images and Embed Videos

Aim	Upload images/videos to WordPress Media Library and embed them into posts and pages.
Duration	1 Lab Session
Tools	WordPress Editor, Media Library, YouTube/Vimeo embed links

Step-by-Step Instructions

Part A: Upload and Insert Images

140. Go to Posts > Add New (or open an existing post).
141. Click the + (Add Block) icon.
142. Search for and select the Image block.
143. Click Upload to upload an image from your computer, or select from Media Library.
144. After inserting, you can set Alt Text, alignment (left/center/right), and size.
145. Alternatively, go to Media > Add New to bulk upload images.

Part B: Insert Image as Featured Image

146. In the post editor, find Set Featured Image in the right panel.
147. Click and upload a banner/thumbnail image.
148. This image appears on blog listings and at the top of the post.

Part C: Embed YouTube Video

149. Open a YouTube video, click Share > Copy Link.
150. In the WordPress editor, click the + icon.
151. Search for YouTube and select the YouTube block (or use the Embed block).
152. Paste the YouTube URL into the field and click Embed.
153. The video player will appear directly in the editor preview.

Part D: Embed Vimeo Video

154. Copy a Vimeo video URL (e.g., https://vimeo.com/123456789).
155. In the editor, use the Embed block and paste the Vimeo URL.
156. Click Embed to display the Vimeo player.

Expected Output: Post contains an uploaded image, a featured image, and at least one embedded YouTube or Vimeo video.

Viva Questions

157. What media file types does WordPress support by default?
158. What is the WordPress Media Library?
159. What is an oEmbed? How does WordPress use it for video embedding?

Practical 13: Create a Gallery of Images

Aim	Create and display an image gallery using the WordPress Gallery block.
Duration	1 Lab Session
Tools	WordPress Editor, Media Library

Step-by-Step Instructions

Step 1: Upload Multiple Images

160. Go to Media > Add New.
161. Upload 6-10 images to your WordPress Media Library.

Step 2: Add Gallery Block to a Page

162. Go to Pages > Add New.
163. Enter a title: 'Photo Gallery'
164. Click the + icon and search for Gallery.
165. Select the Gallery block.

Step 3: Select Images for Gallery

166. Click Upload or Media Library.
167. Select all images you want in the gallery (click each while holding Ctrl/Cmd).
168. Click Create a new gallery.
169. Reorder images by dragging if desired.
170. Click Insert gallery.

Step 4: Configure Gallery Settings

171. In the right panel, set number of Columns (2, 3, or 4).
172. Toggle Crop images to make all images the same size.
173. Choose image size: Thumbnail, Medium, or Large.
174. Enable Link To: Media File (opens full image on click).

Step 5: Publish the Page

175. Click Publish.
176. View the page on the frontend to see the gallery layout.

Expected Output: A clean multi-column image gallery displayed on a WordPress page.

Viva Questions

177. What is the difference between the Gallery block and the Image block?
178. How can you add captions to gallery images in WordPress?
179. What WordPress plugins can enhance gallery functionality?

Practical 14: Install and Activate a New Theme

Aim	Browse, install, activate, and customize a WordPress theme.
Duration	1 Lab Session
Tools	WordPress Admin, Theme Repository

Theory

WordPress themes control the visual appearance and layout of a website. Themes are stored in wp-content/themes/. WordPress.org provides thousands of free themes. Child themes allow you to customize a theme without modifying core theme files.

Step-by-Step Instructions

Step 1: Access Theme Management

180. Login to WordPress Dashboard.
181. Go to Appearance > Themes.
182. You will see currently installed themes.

Step 2: Add New Theme

183. Click Add New Theme at the top.
184. Browse the theme library — use search or filters (Popular, Latest, Feature Filter).
185. Recommended free themes: Astra, OceanWP, Neve, or Kadence.
186. Hover over a theme and click Preview to see a live demo.
187. Click Install on your chosen theme.

Step 3: Activate the Theme

188. After installation, click Activate.
189. Visit the frontend (http://localhost/mysite) to see the new look.

Step 4: Customize the Theme

190. Go to Appearance > Customize.
191. Explore available options: Colors, Typography, Header, Footer, Layout.
192. Make changes and use the live preview to see effects in real time.
193. Click Publish when done.

Step 5: Install Starter Template (Optional for Astra)

194. If using Astra, install the Starter Templates plugin.
195. Import a pre-built template to get a complete page layout instantly.

Expected Output: Website appearance changed to the newly activated theme with custom colors/fonts.

Viva Questions

196. What is a WordPress theme and where are theme files stored?
197. What is a child theme and why is it recommended?
198. What is the difference between a free and a premium WordPress theme?
199. How do you upload and install a theme from a .zip file?

Practical 15: Create New User Accounts with Different Roles

Aim	Create WordPress user accounts with different roles and understand role-based permissions.
Duration	1 Lab Session
Tools	WordPress Admin Dashboard, Users Section

Theory

WordPress has a role-based access control system. The five default roles are: Administrator, Editor, Author, Contributor, and Subscriber. Each role has a defined set of capabilities (permissions) that determine what actions a user can perform on the site.

Role	Capabilities
Administrator	Full control: manage themes, plugins, users, settings, all content
Editor	Manage and publish all posts and pages, moderate comments
Author	Write, edit, and publish their own posts; upload media
Contributor	Write and edit their own posts but cannot publish them
Subscriber	Read content and manage their own profile only

Step-by-Step Instructions

Step 1: Add an Editor User

200. Go to Users > Add New in the Dashboard.

201. Fill in the details:

```
Username: editor_user
Email:    editor@example.com
Password: (generate or enter a strong password)
Role:     Editor
```

202. Click Add New User.

Step 2: Add an Author User

203. Repeat the above process with:

```
Username: author_user
Email:    author@example.com
Role:     Author
```

Step 3: Add a Subscriber User

204. Repeat with:

```
Username: subscriber_user
Email:    subscriber@example.com
Role:     Subscriber
```

Step 4: Test Role Permissions

205. Logout from admin account.
206. Login as editor_user — verify you can manage all posts and pages.
207. Login as author_user — verify you can only manage your own posts.
208. Login as subscriber_user — verify you can only see the dashboard and edit your profile.

Step 5: Change a User's Role

209. Login as admin again.
210. Go to Users > All Users.
211. Select a user, click Edit.
212. Change the Role dropdown and click Update User.

Expected Output: Three new user accounts created. Each account has permissions matching their assigned role.

Viva Questions

213. What are the five default WordPress user roles?
214. What is the difference between Author and Contributor?
215. Can you create custom user roles in WordPress? How?
216. What plugin is commonly used to manage user roles in WordPress?

Practical 16: Create and Deploy Demo Web Application using Laravel (Final Project)

Aim	Design, develop, and deploy a complete project-based web application using Laravel and PHP.
Duration	2-3 Lab Sessions (Project Work)
Tools	Laravel, PHP, MySQL, XAMPP, Bootstrap, Blade

Project Ideas (Choose One)

- Student Result Management System
- Library Book Management System
- Online Quiz/MCQ Application
- Event Registration Portal
- Simple E-Commerce Product Catalog

Project Development Guidelines

Phase 1: Planning

217. Define the project scope and features.
218. Design the database schema (Entity-Relationship Diagram).
219. Identify the modules: Auth, CRUD, Reports, etc.

Phase 2: Setup

```
composer create-project laravel/laravel my-project
cd my-project
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
```

Phase 3: Database Design

220. Create migrations for all required tables.
221. Define Eloquent relationships (hasMany, belongsTo, etc.).
222. Run migrations: php artisan migrate

Phase 4: Build Features

223. Create Models, Controllers, and Resource Routes.
224. Implement authentication using Breeze.

225. Build CRUD operations for core entities.
226. Create Blade views with a consistent layout.
227. Add form validation.

Phase 5: Testing

228. Test all routes and CRUD functionality.
229. Test authentication: register, login, logout.
230. Test role-based access if implemented.
231. Check for broken links and errors.

Phase 6: Deploy on XAMPP

232. Copy the project to C:\xampp\htdocs\my-project\
233. Set correct APP_ENV and APP_DEBUG in .env
234. Run php artisan config:cache and php artisan route:cache
235. Access the app at http://localhost/my-project/public

Deliverables

- Source code of the complete Laravel application
- Database migration files and sample data (seeders)
- Screenshots of all working pages
- Brief report describing features and implementation

Expected Output: A complete, deployed, and working web application built with Laravel and PHP.

Viva Questions

236. Explain the complete request lifecycle in Laravel.
237. What are Laravel Service Providers?
238. What is the difference between php artisan serve and XAMPP deployment?
239. How do you secure a Laravel application before deployment?
240. What are Laravel seeders and factories? When would you use them?

Appendix: Quick Reference

Useful Laravel Artisan Commands

Command	Purpose
`php artisan serve`	Start development server
`php artisan make:model Name -mcr`	Create Model, Migration, Controller
`php artisan migrate`	Run all pending migrations
`php artisan migrate:rollback`	Undo last migration batch
`php artisan route:list`	List all registered routes
`php artisan make:controller Name`	Create a controller
`php artisan make:migration name`	Create a new migration file
`php artisan config:cache`	Cache config for production
`php artisan db:seed`	Run database seeders

WordPress Admin Quick Reference

- Posts > Add New — Create a blog post
- Pages > Add New — Create a static page
- Appearance > Themes — Manage themes
- Appearance > Customize — Live site customization
- Media > Library — Manage uploaded files
- Users > Add New — Create user accounts
- Settings > General — Site title, tagline, URL
- Plugins > Add New — Install plugins

Marking Scheme

Component	Internal (20)	External (30)
Practical Execution & Code	10	15
Viva / Lab Quiz	5	10
Journal / Record Work	5	5
Total	**20**	**30**